

# Python pour le trading — Document 6/8

L'interaction avec le monde extérieur

## 1. Les requêtes HTTP

Le **HTTP** est le protocole d'échange du web. Beaucoup d'API de marché s'interrogent par des requêtes HTTP : on demande une information (le dernier prix, l'historique), le serveur répond. En Python, la bibliothèque `requests` est la plus simple.

```
import requests

# GET : demander une information
reponse = requests.get("https://api.exemple.com/prix/ES")
reponse.status_code      # 200 = succès, 404 = introuvable, etc.
donnees = reponse.json() # convertir la réponse JSON en dict Python

# POST : envoyer une information (par ex. passer un ordre)
ordre = {"symbole": "ES", "quantite": 5, "prix": 5234.50}
reponse = requests.post("https://api.exemple.com/ordres", json=ordre)
```

Une requête GET **demande** une donnée ; une requête POST **envoie** une donnée (par exemple un ordre). Le `status_code` indique si la requête a réussi (200) ou échoué (404, 500...). `reponse.json()` convertit la réponse en dictionnaire Python exploitable. C'est le mécanisme de base pour les API « requête-réponse », adaptées aux données qu'on consulte ponctuellement.

### 1.1 Gérer les erreurs réseau

Un appel réseau peut échouer : serveur lent, indisponible, réponse en erreur. Du code de production ne suppose jamais que tout se passe bien. Trois réflexes : un `timeout` pour ne pas attendre indéfiniment, `raise_for_status()` pour transformer un code 4xx/5xx en exception, et un `try / except` pour réagir proprement.

```
import requests

try:
    reponse = requests.get("https://api.exemple.com/prix/ES", timeout=5)
    reponse.raise_for_status()      # lève une exception si status >= 400
    prix = reponse.json()["prix"]
except requests.Timeout:
    print("Délai dépassé : serveur trop lent")
except requests.HTTPError as err:
    print(f"Réponse en erreur : {err}")
except requests.RequestException as err:
    print(f"Problème réseau : {err}")

timeout=5 interrompt l'attente après cinq secondes. raise_for_status convertit une réponse 404 ou 500 en exception, qu'on attrape comme le reste. RequestException est la classe parente : elle rattrape tout problème réseau resté non traité. Sur un flux d'ordres, ignorer ces cas, c'est risquer de croire un envoi réussi alors qu'il a échoué.
```

**Pourquoi ça compte.** Récupérer un historique, consulter un solde, passer un ordre par API REST : autant d'opérations HTTP. Maîtriser GET/POST et la lecture du `status_code` est la base de toute connexion à une plateforme.

## 2. Les WebSockets : le temps réel

Le HTTP fonctionne en « demande puis réponse », ce qui est inadapté pour un flux continu. Pour recevoir les prix **en temps réel**, on utilise un **WebSocket** : une connexion permanente où le serveur **pousse** les données dès qu'elles arrivent, sans qu'on ait à redemander.

```
import websockets
import asyncio

async def suivre_prix():
    url = "wss://flux.exemple.com/marche"
    async with websockets.connect(url) as ws:      # connexion permanente
        await ws.send('{"abonnement": "ES"}')    # on s'abonne à un flux
        async for message in ws:                 # on reçoit en continu
            traiter(message)                      # chaque prix dès son arrivée

asyncio.run(suivre_prix())
```

La différence avec HTTP est fondamentale. Avec `websockets.connect`, on ouvre une connexion qui reste ouverte. On s'abonne à un flux, puis `async for message in ws` reçoit chaque nouvelle donnée dès que le serveur l'envoie — pas besoin de redemander. On reconnaît `async` et `await` du document 4 : les WebSockets vont de pair avec `asyncio`, l'outil privilégié pour suivre plusieurs flux ; mais le `threading` convient également, puisqu'il s'agit d'I/O-bound. C'est le mécanisme des données de marché en direct.

**Pourquoi ça compte.** Reconstruire un *order flow* en temps réel, repose sur les WebSockets : le serveur pousse chaque tick instantanément. C'est la fondation de toute surveillance de marché en direct, au cœur du poste de trade operations.

### 2.1 La même chose en threading

`asyncio` n'est pas la seule option. Comme suivre un flux est de l'I/O-bound (on passe le temps à attendre le réseau, pas à calculer), le **threading** convient tout aussi bien. C'est même plus pratique quand le reste du programme n'est pas asynchrone : on isole le suivi dans un thread, et le programme principal continue de tourner.

```
import websocket          # bibliothèque « websocket-client » (synchrone)
import threading

def suivre_prix():
    ws = websocket.create_connection("wss://flux.exemple.com/marche")
    ws.send('{"abonnement": "ES"}')    # on s'abonne au flux
    while True:                        # boucle bloquante
        message = ws.recv()            # attend le prochain message
        traiter(message)

# on lance le suivi dans un thread pour ne pas bloquer le programme
thread = threading.Thread(target=suivre_prix, daemon=True)
```

```
thread.start()
```

Ici, `create_connection` ouvre la connexion permanente, et `recv()` attend le message suivant en bloquant le thread. Comme ce code tourne dans un thread séparé, le reste du programme (interface, autres calculs) continue. Pour suivre plusieurs symboles, on lance un thread par connexion.

Quelle approche choisir ? `asyncio` est plus économe : un seul thread suit des centaines de connexions. Le `threading` est plus simple à greffer sur du code existant non asynchrone, mais chaque connexion consomme un thread — parfait pour quelques flux, moins pour des centaines. Les deux sont corrects, car la tâche est I/O-bound (document 4).

### 3. Le JSON : le format d'échange

Le **JSON** est le format texte universel pour échanger des données structurées entre systèmes. Presque toutes les API parlent JSON. Python le convertit facilement vers et depuis ses dictionnaires.

```
import json

# D'un objet Python VERS du texte JSON (sérialiser)
ordre = {"symbole": "ES", "quantite": 5, "prix": 5234.50}
texte = json.dumps(ordre)
# -> '{"symbole": "ES", "quantite": 5, "prix": 5234.5}'

# Du texte JSON VERS un objet Python (désérialiser)
recu = '{"symbole": "NQ", "quantite": 2}'
donnees = json.loads(recu)
donnees["symbole"]      # -> "NQ"
```

Deux opérations symétriques : `json.dumps` transforme un objet Python en texte JSON (pour l'envoyer), `json.loads` fait l'inverse (pour lire ce qu'on reçoit). On parle de **sérialisation** et de **désérialisation**. C'est ainsi qu'un ordre devient un message transmissible, et qu'un message reçu redevient un dictionnaire exploitable. Le moyen mnémotechnique : « s » comme `dumps` pour *string* (vers le texte).

#### 3.1 La structure d'un message JSON

Un message JSON repose sur deux structures et quelques types simples. Les **objets**, entre accolades `{ }`, associent des clés à des valeurs — ils deviennent des dictionnaires Python. Les **tableaux**, entre crochets `[ ]`, listent des valeurs ordonnées — ils deviennent des listes. À l'intérieur, on trouve des chaînes, des nombres, des booléens (`true/false`) et `null`.

```
# Un message reçu d'une plateforme : un objet contenant un tableau d'objets
message = """
{
  "symbole": "ES",
  "horodatage": 1700000000,
  "actif": true,
  "meilleures_offres": [
    {"prix": 5234.50, "quantite": 12},
    {"prix": 5234.75, "quantite": 8}
  ],
  "note": null
}
"""
carnet = json.loads(message)
```

```
carnet["meilleures_offres"][0]["prix"] # -> 5234.5
```

La correspondance est directe : un objet {} devient un dict, un tableau [] une list, une chaîne un str, un nombre un int ou un float, true/false des booléens True/False, et null la valeur None. On navigue ensuite dans le résultat comme dans n'importe quelle structure Python imbriquée.

### 3.2 Sérialiser différents types

`json.dumps` ne se limite pas à un dictionnaire plat : il sérialise aussi les listes et les valeurs imbriquées, et accepte des options utiles pour produire une sortie lisible.

```
# Une liste d'ordres, mise en forme lisible
ordres = [
    {"symbole": "ES", "quantite": 5},
    {"symbole": "NQ", "quantite": 2},
]
print(json.dumps(ordres, indent=2, ensure_ascii=False))
# indent=2 -> sortie indentée et lisible
# ensure_ascii=False -> garde les accents (é, à) tels quels
```

Attention : tous les types Python ne sont pas sérialisables directement. Un objet `datetime`, par exemple, déclenche une erreur ; il faut le convertir en chaîne (souvent au format ISO) avant l'appel. À l'inverse, `json.loads` ne rend que des types simples : une date relue redevient une chaîne, jamais un `datetime`. C'est une source classique d'écarts entre systèmes.

**Pourquoi ça compte.** Chaque message échangé avec une plateforme — ordre envoyé, prix reçu — transite en JSON. Savoir le sérialiser et le désérialiser proprement est indispensable, et c'est aussi là que surviennent les écarts de données à résoudre (formats divergents entre sources).

## 4. Les bases de données

Une **base de données** stocke durablement les données de façon structurée et interrogeable. L'offre Tower mentionne le reporting et la réconciliation, qui s'appuient sur des bases. Python s'y connecte, et le langage d'interrogation standard est le **SQL**.

```
import sqlite3

conn = sqlite3.connect("trading.db")
curseur = conn.cursor()

# Interroger : récupérer les ordres sur ES
curseur.execute("SELECT symbole, prix FROM ordres WHERE symbole = ?", ("ES",))
lignes = curseur.fetchall() # -> liste des résultats

# Insérer un nouvel ordre
curseur.execute("INSERT INTO ordres (symbole, prix) VALUES (?, ?)",
                ("NQ", 18450.0))
conn.commit() # valider l'écriture
conn.close()
```

On ouvre une connexion, on envoie des requêtes SQL via un curseur. `SELECT ... WHERE` récupère des données, `INSERT` en ajoute. Détail crucial : les `?` sont des **paramètres**, et la valeur est passée séparément. C'est essentiel pour la **sécurité** (cela évite les injections SQL) et la fiabilité. `commit` valide définitivement une écriture. C'est le socle du stockage et de la réconciliation de positions.

#### 4.1 Anatomie d'une session SQLite

Une session suit toujours les mêmes étapes. `connect` ouvre la base — ici un simple fichier `trading.db`, créé s'il n'existe pas encore. Le **curseur** est l'objet qui exécute les requêtes et retient leurs résultats. `execute` lance une requête SQL ; `fetchall` récupère toutes les lignes d'un `SELECT` (ou `fetchone` pour la première). `commit` valide les écritures, et `close` referme proprement.

Avant d'insérer, encore faut-il que la table existe. On la crée une seule fois :

```
# Créer la table si elle n'existe pas encore
curseur.execute("""
    CREATE TABLE IF NOT EXISTS ordres (
        id          INTEGER PRIMARY KEY,
        symbole     TEXT,
        prix        REAL
    )
""")
conn.commit()
```

`fetchall` rend une liste de tuples ; on peut aussi parcourir directement le curseur, qui livre les lignes une à une :

```
for symbole, prix in curseur.execute("SELECT symbole, prix FROM ordres"):
    print(symbole, prix)
```

#### 4.2 Les paramètres ? et l'injection SQL

Pourquoi écrire `VALUES (?, ?)` plutôt que de coller la valeur directement dans le texte de la requête ? Parce que coller une valeur venue de l'extérieur est dangereux.

```
# DANGEREUX : la valeur est collée dans le texte de la requête
symbole = entree_utilisateur      # ex. : "ES"; DROP TABLE ordres; --"
curseur.execute(f"SELECT * FROM ordres WHERE symbole = '{symbole}'")
```

```
# SÛR : la valeur est passée séparément, jamais interprétée comme du SQL
curseur.execute("SELECT * FROM ordres WHERE symbole = ?", (symbole,))
```

Dans la première version, une valeur malveillante peut détourner la requête et, ici, effacer la table : c'est une **injection SQL**. Dans la seconde, le `?` est un simple emplacement : la base traite la valeur comme une donnée, jamais comme du code. Règle absolue : toute valeur variable passe par un paramètre.

#### 4.3 De la base vers Pandas

Pour analyser ou réconcilier, on charge souvent le résultat d'une requête directement dans un `DataFrame` (document 5). Pandas lit une requête SQL en une ligne :

```
import pandas as pd

# Charger le résultat d'une requête dans un DataFrame
df = pd.read_sql_query("SELECT * FROM ordres WHERE symbole = ?",
                       conn, params=("ES",))
```

```
# ... analyse, réconciliation ...
df.to_parquet("ordres_es.parquet") # archiver pour plus tard
```

read\_sql\_query exécute la requête et range les lignes dans un DataFrame, prêt pour le tri, l'agrégation ou la comparaison avec un relevé. C'est le pont entre la base (stockage) et Pandas (analyse), au cœur d'un pipeline API → base → fichier.

**Pourquoi ça compte.** Le reporting PnL, l'historique des transactions et la réconciliation avec les relevés de courtiers reposent sur des bases de données. Savoir interroger et écrire en SQL, de façon sûre (paramètres), est directement lié aux responsabilités du poste.

## 5. La lecture et l'écriture de fichiers

Les données circulent aussi sous forme de **fichiers** : un relevé de courtier en CSV, un historique exporté, une configuration. On lit et écrit ces fichiers en s'appuyant sur le context manager `with` du document 3, qui garantit leur fermeture.

### 5.1 CSV avec Pandas

```
import pandas as pd

# Lire un relevé de courtier
df = pd.read_csv("releve_courtier.csv")

# ... traitement, réconciliation ...

# Écrire le résultat
df.to_csv("reconciliation.csv", index=False)
```

Pour des données tabulaires, Pandas (document 5) lit un CSV en une ligne avec `read_csv` et l'écrit avec `to_csv`. C'est la façon la plus directe de charger un relevé de courtier pour le comparer aux positions internes — exactement le type de tâche de réconciliation du poste.

### 5.2 Le format Parquet pour les gros volumes

```
# Parquet : format compact et rapide pour de gros historiques
df.to_parquet("historique_es.parquet")
df = pd.read_parquet("historique_es.parquet")
```

Pour de gros historiques de marché, le format **Parquet** est préférable au CSV : il est compressé et bien plus rapide à lire et écrire, car il stocke les données par colonnes. C'est le format de choix quand les volumes deviennent importants — le cas typique des données de marché.

**Pourquoi ça compte.** Charger un relevé pour le réconcilier, sauvegarder un historique pour l'analyser : la manipulation de fichiers est quotidienne. CSV pour les échanges simples, Parquet pour les gros volumes — le bon format évite des lenteurs inutiles.

## 6. Les secrets : clés d'API et identifiants

Pour se connecter à une plateforme — par HTTP ou WebSocket — il faut des identifiants : une clé d'API, parfois un secret associé. Première règle, absolue : on ne les écrit **jamais en dur** dans le code.

```
# À NE PAS FAIRE : clé en dur dans le code
cle = "sk_live_8f3a9c2b..." # exposée dès que le code est partagé
```

```
# BIEN : lire la clé depuis une variable d'environnement
import os
cle = os.environ["CLE_API_COURTIER"] # définie hors du code
```

La clé est définie dans l'environnement (une variable du système, ou un fichier `.env` non versionné) ; le code se contente de la lire. La même base de code fonctionne alors en test et en production, sans qu'aucun secret n'y figure. `os.getenv` offre une variante qui ne plante pas si la variable est absente, et permet un message clair :

```
cle = os.getenv("CLE_API_COURTIER")
if cle is None:
    raise RuntimeError("Clé d'API manquante : définir CLE_API_COURTIER")
```

```
reponse = requests.get(url, headers={"Authorization": f"Bearer {cle}"})
```

En pratique, on range ces variables dans un fichier `.env`, exclu du dépôt par le `.gitignore` (document 7) et chargé au démarrage (par exemple avec `python-dotenv`). La règle tient en une phrase : jamais de secret dans le code, jamais de secret dans Git.

**Pourquoi ça compte.** Une clé d'API qui fuite, c'est un accès direct au compte de trading. Les gérer par variables d'environnement, hors du code et hors du dépôt, est une exigence de sécurité de base pour toute connexion à une plateforme.

## Récapitulatif

1. **HTTP (requests)** : GET pour demander, POST pour envoyer ; vérifier le `status_code`.
2. **WebSockets** : connexion permanente où le serveur pousse les données en temps réel ; va de pair avec `asyncio` (le `threading` convient aussi, l'I/O étant le facteur limitant).
3. **JSON** : le format d'échange ; `dumps` pour sérialiser, `loads` pour désérialiser.
4. **Bases de données (SQL)** : SELECT/INSERT via un curseur ; utiliser des paramètres (?) pour la sécurité ; `commit` pour valider.
5. **Fichiers** : `read_csv/to_csv` pour le tabulaire, `Parquet` pour les gros volumes ; toujours via `with`.
6. **Secrets** : clés d'API via des variables d'environnement (`os.environ` / `os.getenv`) ; jamais en dur dans le code ni dans Git.

## Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

### Sur HTTP et les WebSockets

**Q1.** Quelle est la différence entre une requête GET et une requête POST ?

**Réponse.** GET demande une information au serveur ; POST envoie une information au serveur (par exemple pour passer un ordre).

**Q1b.** Comment rendre un appel HTTP robuste aux erreurs réseau ?

**Réponse.** Avec un timeout (éviter l'attente infinie), `raise_for_status()` (transformer un code 4xx/5xx en exception) et un `try/except` pour réagir proprement.

**Q2.** Pourquoi le HTTP est-il inadapté à un flux de prix en temps réel ?

**Réponse.** Parce qu'il fonctionne en demande-réponse : il faudrait redemander sans cesse. Un WebSocket maintient une connexion ouverte où le serveur pousse les données dès qu'elles arrivent.

**Q3.** Qu'est-ce qu'un WebSocket et avec quel outil va-t-il de pair ?

**Réponse.** Une connexion permanente où le serveur envoie les données dès qu'elles arrivent, sans redemander. `asyncio` est l'outil privilégié pour suivre de nombreux flux, mais le `threading` convient également puisqu'il s'agit d'I/O-bound.

**Q3b.** Peut-on suivre un WebSocket sans `asyncio` ?

**Réponse.** Oui : suivre un flux est de l'I/O-bound, donc le `threading` convient aussi. On isole le suivi dans un thread et le programme continue. `asyncio` reste préférable pour un grand nombre de flux simultanés.

## Sur le JSON

**Q4.** Que font `json.dumps` et `json.loads` ?

**Réponse.** `dumps` sérialise : il transforme un objet Python en texte JSON (pour l'envoyer). `loads` désérialise : il transforme du texte JSON reçu en objet Python.

**Q5.** Pourquoi le JSON est-il important pour un système de trading ?

**Réponse.** Parce que chaque message échangé avec une plateforme (ordre envoyé, prix reçu) transite en JSON. C'est aussi un endroit où surviennent des écarts de données dus à des formats divergents entre sources.

**Q5b.** À quoi correspondent un objet `{ }` et un tableau `[ ]` JSON une fois lus en Python ?

**Réponse.** Un objet devient un dict, un tableau devient une list. Les chaînes, nombres, booléens et null deviennent `str`, `int/float`, `True/False` et `None`.

## Sur les bases de données

**Q6.** Quelle est la différence entre `SELECT` et `INSERT` en SQL ?

**Réponse.** `SELECT` récupère des données existantes ; `INSERT` ajoute de nouvelles données dans la table.

**Q7.** Pourquoi passer les valeurs via des paramètres (?) plutôt que de les insérer dans la requête ?

**Réponse.** Pour la sécurité (éviter les injections SQL) et la fiabilité. La valeur est passée séparément de la requête, jamais collée dans le texte.

**Q8.** À quoi sert `commit` ?

**Réponse.** À valider définitivement une écriture (`INSERT`, `UPDATE`) dans la base. Sans `commit`, la modification n'est pas enregistrée.

**Q8b.** À quoi sert le curseur, et que renvoie `fetchall` ?



**Réponse.** Le curseur exécute les requêtes et retient leurs résultats. fetchall renvoie toutes les lignes d'un SELECT, sous forme de liste de tuples.

**Q8c.** Comment charger le résultat d'une requête SQL dans Pandas ?

**Réponse.** Avec `pd.read_sql_query(requête, conn, params=...)`, qui range directement le résultat dans un DataFrame.

## Sur les fichiers

**Q9.** Avec quoi lit-on et écrit-on un CSV en Pandas ?

**Réponse.** `read_csv` pour lire, `to_csv` pour écrire. C'est la façon la plus directe de charger un relevé de courtier ou de sauvegarder un résultat.

**Q10.** Pourquoi préférer Parquet au CSV pour de gros historiques ?

**Réponse.** Parquet est compressé et bien plus rapide à lire et écrire, car il stocke les données par colonnes. C'est le format de choix pour les gros volumes de données de marché.

## Sur les secrets

**Q11.** Où stocker une clé d'API, et pourquoi pas dans le code ?

**Réponse.** Dans une variable d'environnement (lue avec `os.environ` ou `os.getenv`), définie hors du code et hors du dépôt. En dur dans le code, elle fuiterait dès que le code est partagé ou versionné — donnant un accès direct au compte.

*Document suivant (7/8) : la qualité, le test et l'outillage — pytest, mocking, logging, débogage et profilage de performance, ce qui distingue du code personnel d'un code de production.*